

Pypomp: Inference for partially observed Markov process models in Python with JAX DRAFT IN PROGRESS

Aaron J. Abkemeier¹

Jun Chen²

Kevin Tan³

Jesse Wheeler⁴

Bo Yang⁵

Kunyang He¹

Jonathan Terhorst¹

Aaron A. King⁶

Edward L. Ionides¹

AARONABK@UMICH.EDU

CHENJUNC@UMICH.EDU

KEVTAN@WHARTON.UPENN.EDU

JESSEWHEELER@ISU.EDU

YBB@UMICH.EDU

KUNYANGHE@UMICH.EDU

JONTH@UMICH.EDU

KINGAA@UMICH.EDU

IONIDES@UMICH.EDU

¹University of Michigan, Department of Statistics; ²Duke University, Department of Statistics;
³University of Pennsylvania, Department of Statistics; ⁴Idaho State University, Department of
Mathematics; ⁵Columbia University, Department of Marketing; ⁶University of Michigan, Depart-
ment of Ecology and Evolutionary Biology

Editor:

Abstract

Particle filtering methods provide statistically efficient inference for nonlinear, non-Gaussian partially observed Markov process (POMP) models, which are widely used in epidemiology, ecology, and finance, but their computational demands limit their practical use. We introduce Pypomp, a Python library for inference on POMP models that uses JAX to take advantage of GPU hardware, automatic differentiation, and just-in-time compilation. On one GPU, iterated filtering can run with the throughput of more than 8,000 CPU cores running the R package pomp. Pypomp’s interface streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially observed Markov process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling dynamic systems whose underlying state is not directly observed. The particle filter (Arulampalam et al., 2002), also known as sequential Monte Carlo, estimates the likelihood of a POMP model, and iterated filtering (Ionides et al., 2006, 2015) extends it to maximum likelihood estimation. These methods only need to simulate from the process model, not evaluate its transition probabilities, a property known as plug-and-play (Bretó et al., 2009). Consequently, they apply to a wide range of models unconstrained by mathematical convenience, and they are used in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al.,

2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where mechanistic dynamic modeling is desired.

Pypomp is a Python library combining advances in hardware, software, and algorithms to provide novel capabilities for POMP models, with an emphasis on inference via particle filters. It is built on JAX (Bradbury et al., 2018), which provides just-in-time (JIT) compilation, automatic differentiation (AD), and access to massively parallel hardware such as GPUs. Pypomp inherits its software design from the R package `pomp` (King et al., 2016) and implements the algorithmic advances needed to take advantage of JAX. For AD, it implements a differentiable particle filter that controls both bias and variance (Tan et al., 2024). Rejection sampling algorithms for common distributions are poorly suited to GPUs, so Pypomp also provides alternative samplers (Giles, 2016; Giles and Beentjes, 2024; Temme, 1992).

When the state space is small and discrete, or the model is linear Gaussian or close enough for Gaussian filtering approximations, the particle filter is unnecessary, and the Dynamax library (Linderman et al., 2025) provides a JAX-based solution. Pypomp is aimed at more complex, nonlinear models such as those common in ecology and epidemiology, although its plug-and-play property also makes it convenient for rapid model development even when alternative methods apply. It also supports panel POMP models and methods, including panel iterated filtering (PIF) and marginalized panel iterated filtering (MPIF) (Bretó et al., 2020; Wheeler et al., 2025).

With enough particles and method replications to saturate it, one GPU runs iterated filtering as fast as 3,152 to 8,659 CPU cores running `pomp` or `panelPomp` (Appendix B). Such replications are routine in practice: likelihood maximization involves searching from many initial parameter vectors, and Monte Carlo adjusted profile (MCAP) confidence intervals (Ionides et al., 2017), which Pypomp computes with `mcap`, require many iterated filtering searches at each point of a likelihood profile. The GPU also makes plug-and-play inference feasible at previously intractable scales: one GPU fits a panel POMP model to the England and Wales measles data of Korevaar et al. (2020), over one million observations from 1,422 towns and cities, in 29 hours, compared with an estimated 4.5 months for `panelPomp` on a 36-core node (Section 3).

2 POMP Modeling in Pypomp

A POMP model consists of an unobserved Markov process $\{X(t), t_0 \leq t \leq t_N\}$ and observations $Y_{1:N}$ made at times $t_{1:N}$, with observed values $y_{1:N}^*$. The initial state is defined at time t_0 , $X_n := X(t_n)$, and Y_n is independent of Y_m, X_m , $m \neq n$, conditional on X_n . Plug-and-play methods need only a simulator for the initial state, $X_0 \sim f_{X_0}(x_0; \theta)$, a simulator for the state transition, $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n; \theta)$, and the measurement density $f_{Y_n|X_n}(y_n|x_n; \theta)$, where θ denotes the model parameters. In Pypomp, the user passes the data and programmatic descriptions of these functions to a `Pomp` object.

Pypomp implements the particle filter (`pfilter`) for likelihood evaluation and state estimation, iterated filtering 2 (IF2) (`mif`) for global maximum likelihood estimation, and the differentiable particle filter MOP- α (`mop`, Tan et al. (2024)), whose gradients the `train` method uses for local optimization. These are methods of the `Pomp` class, and each model

component is a JAX-compatible function. For example, the S&P 500 model included in the package defines its state transition simulator as follows:

```
import jax
import jax.numpy as jnp
from pypomp.types import (StateDict, ParamDict, CovarDict,
                          TimeFloat, StepSizeFloat, RNGKey)

def rproc(X_: StateDict, theta_: ParamDict, key: RNGKey,
          covars: CovarDict, t: TimeFloat, dt: StepSizeFloat):
    V, S = X_["V"], X_["S"]
    mu, kappa, theta = theta_["mu"], theta_["kappa"], theta_["theta"]
    xi, rho, y_prev = theta_["xi"], theta_["rho"], covars["y_prev"]
    dZ = jax.random.normal(key)
    dWs = (y_prev - mu + 0.5 * V) / jnp.sqrt(V)
    dWv = rho * dWs + jnp.sqrt(1 - rho**2) * dZ
    S = S + S * (mu + jnp.sqrt(jnp.maximum(V, 1e-32))) * dWs
    V = V + kappa * (theta - V) + xi * jnp.sqrt(V) * dWv
    V = jnp.maximum(V, 1e-32)
    return {"V": V, "S": S}
```

Pypomp vectorizes such a function over particles and over replicate runs, such as when starting searches from many values of θ , and JIT compiles it so that the computation runs in parallel across GPU cores. The following example searches globally with IF2, refines the estimate by gradient descent, and evaluates the likelihood of the result:

```
import pypomp as pp
spx_obj = pp.models.spx()
key = jax.random.key(1)
rw_sd = pp.RWSigma( # Random walk standard deviations for IF2
    sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1}, init_names=["V_0"],
).geometric_cooling(a=0.5)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, key=key) # IF2
eta = pp.LearningRate(
    {name: 0.001 for name in spx_obj.canonical_param_names}
).geometric_decay(decay_rate=0.5, M=10) # Learning rates
spx_obj.train(J=1000, M=10, eta=eta, optimizer=pp.Adam()) # Gradient descent
spx_obj.pfilter(J=1000, reps=10) # Particle filter
```

Each method call stores its log-likelihood estimates, parameter traces, execution time, and arguments in `spx_obj.results_history`, and helpers such as `traces` return these in convenient formats such as a data frame. The JIT-compiled functions behind these methods are available in `pypomp.functional` for building custom inference procedures. The source code is available at <https://github.com/pypomp/pypomp>, documentation at <https://pypomp.readthedocs.io>, and tutorials that build complete models at <https://pypomp.readthedocs.io>.

Table 1: Wall-clock time in seconds for IF2, running 36 searches from the same starting points. Speedup is the R time divided by the GPU time, and cores matched is the speedup times 36. Peak is the highest value of cores matched in GPU scaling tests with more particles in flight (Appendix B).

Model	R (36 cores)	Pypomp (1 GPU)	Speedup	Cores matched	Peak
Cholera, Dhaka	1,566	28.8	54×	1,959	3,152
Measles, London	2,923	32.2	91×	3,264	5,499
Measles, 4-town panel	8,187	98.2	83×	3,003	3,917
S&P 500	703	34.8	20×	727	8,659

[//pypomp.github.io/tutorials](https://pypomp.github.io/tutorials). The quantitative tests that validate the correctness and speed of Pypomp’s main methods are available at <https://github.com/pypomp/quant>.

3 Performance

We benchmarked Pypomp 1.0.5 against **pomp** 6.3 and **panelPomp** 1.7.0.0 on four models using the quantitative test suite (commit a8a519a). The models differ in the cost of the process model, the number of process model steps between resampling, and whether they are panels, so together they show where the GPU helps most and least (Appendix A). Table 1 compares IF2 wall-clock times for the same workloads with R on a 36-core CPU node and Pypomp on one RTX PRO 6000 Blackwell Server Edition GPU. These workloads are sized for R, and they keep only 36,000 to 180,000 particles in flight, too few to saturate the GPU. In scaling tests that add more searches or more particles per search (Appendix B), throughput keeps rising until 1.0 to 2 million particles are in flight. At that point, one GPU matches 3,152 to 8,659 CPU cores running R, the Peak column of Table 1. The S&P 500 model gains the most from scaling because its benchmark, with 1,000 particles per search and a cheap process model, gives the GPU the least work. The GPU times rely on the approximate samplers in **pypomp.random**: with the standard **jax.random** samplers, IF2 for the London measles model took 46 times longer (Appendix C). Fitting the measles model of He et al. (2010) to all 1,422 towns and cities for 1950–1963 took one GPU 29 hours, including likelihood evaluation, versus an estimated 4.5 months for R on a 36-core node, extrapolating linearly from **panelPomp**’s cost on the 4-town panel (Appendix D).

4 Conclusion

The computational cost of particle filtering has limited both the size of datasets that practitioners can fit and how many models and likelihood profiles they can afford to evaluate. We expect Pypomp’s speed will allow practitioners to iterate more quickly on model structure, fit datasets as large as the England and Wales measles data, and develop and evaluate more realistic models of the systems they study.

References

- M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, February 2002. ISSN 1053587X. doi: 10.1109/78.978374.
- Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470. URL <https://doi.org/10.1002/ecm.1470>.
- J. C. Blackwood, D. G. Streicker, S. Altizer, and P. Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110(51):20837–20842, December 2013. doi: 10.1073/pnas.1308817110. URL <https://doi.org/10.1073/pnas.1308817110>.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.
- Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003. URL <https://doi.org/10.1016/j.spl.2014.04.003>.
- Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, March 2009. ISSN 1932-6157, 1941-7330. doi: 10.1214/08-AOAS201.
- Carles Bretó, Edward L. Ionides, and Aaron A. King. Panel Data Analysis via Mechanistic Models. *Journal of the American Statistical Association*, 115(531):1178–1188, June 2020. ISSN 0162-1459, 1537-274X. doi: 10.1080/01621459.2019.1604367.
- S. J. Fox, M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, T. A. Ingle, E. Javan, M. Dahan, K. Gaither, M. E. Escott, S. I. Adler, S. C. Johnston, J. G. Scott, and L. A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119. URL <https://doi.org/10.1073/pnas.2111870119>.
- Michael B. Giles. Algorithm 955: Approximation of the Inverse Poisson Cumulative Distribution Function. *ACM Transactions on Mathematical Software*, 42(1):1–22, March 2016. ISSN 0098-3500, 1557-7295. doi: 10.1145/2699466. URL <https://dl.acm.org/doi/10.1145/2699466>.
- Michael B. Giles and Casper Beentjes. Approximation of an Inverse of the Incomplete Beta Function. In Kevin Buzzard, Alicia Dickenstein, Bettina Eick, Anton Leykin, and Yue Ren, editors, *Mathematical Software – ICMS 2024*, volume 14749, pages 207–214.

- Springer Nature Switzerland, Cham, 2024. ISBN 978-3-031-64528-0 978-3-031-64529-7. doi: 10.1007/978-3-031-64529-7_22. URL https://link.springer.com/10.1007/978-3-031-64529-7_22. Series Title: Lecture Notes in Computer Science.
- Daihai He, Edward L. Ionides, and Aaron A. King. Plug-and-play inference for disease dynamics: Measles in large and small populations as a case study. *Journal of The Royal Society Interface*, 7(43):271–283, February 2010. ISSN 1742-5689, 1742-5662. doi: 10.1098/rsif.2009.0151.
- E. L. Ionides, C. Breto, and A. A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, December 2006. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0603181103.
- E. L. Ionides, C. Breto, J. Park, R. A. Smith, and A. A. King. Monte Carlo profile confidence intervals for dynamic systems. *Journal of The Royal Society Interface*, 14(132):20170126, July 2017. ISSN 1742-5689, 1742-5662. doi: 10.1098/rsif.2017.0126.
- Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, January 2015. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1410597112.
- Aaron A. King, Edward L. Ionides, Mercedes Pascual, and Menno J. Bouma. Inapparent infections and cholera dynamics. *Nature*, 454(7206):877–880, August 2008. ISSN 1476-4687. doi: 10.1038/nature07084.
- Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the R Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. ISSN 1548-7660. doi: 10.18637/jss.v069.i12.
- Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, July 2020. doi: 10.1098/rsif.2020.0010.
- Scott W Linderman, Peter Chang, Giles Harper-Donnelly, Aleyna Kara, Xinglong Li, Gerardo Duran-Martin, and Kevin Murphy. Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, 10(108):7069, 2025. doi: 10.21105/joss.07069.
- J. A. Jr. Marino, S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, March 2019. doi: 10.1002/ecy.2583. URL <https://doi.org/10.1002/ecy.2583>.
- Matthew S. Mitchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating sars-cov-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, Jan 2024. URL <https://doi.org/10.1101/2024.01.10.24301116>.

Kevin Tan, Giles Hooker, and Edward L. Ionides. Accelerated Inference for Partially Observed Markov Processes using Automatic Differentiation, July 2024.

N M Temme. Asymptotic Inversion of Incomplete Gamma Functions. April 1992.

L. Wen, Y. Yin, Q. Li, Z. Peng, and D. He. Modeling the co-circulation of influenza and covid-19 in hong kong, china. *Advances in Continuous and Discrete Models*, 2024 (1):1–9, 2024. doi: 10.1186/s13662-024-03830-7. URL <https://doi.org/10.1186/s13662-024-03830-7>. Article 32.

Jesse Wheeler, Aaron J. Abkemeier, and Edward L. Ionides. Iterating marginalized Bayes maps for likelihood maximization with application to nonlinear panel models, 2025. URL <https://arxiv.org/abs/2511.17438>.

Appendix A. Full benchmark results

The four benchmark models stress different parts of the particle filter, which helps explain why the GPU’s advantage varies across them.

Cholera, Dhaka. The cholera model of King et al. (2008) has 600 monthly observations with 20 Euler steps between each, and each step updates a stochastic differential equation with one normal draw. The process model is cheap, but there are many steps between resamples, so this model isolates the per-step overhead inside the process model.

Measles, London. The measles model of He et al. (2010) has weekly case reports from 1950 to 1963 and takes a daily Euler step, interpolating covariates and drawing binomial, gamma, and Poisson random variables at every step. It has the most expensive process model of the four, so it isolates how the GPU handles heavy computation per particle, and it relies on the approximate samplers in `pypomp.random` (Appendix C).

Measles, 4-town panel. The panel model fits the same measles model to London, Halesworth, Hastings, and Cardiff jointly, with some parameters shared across towns and the rest specific to each town. MPIF (Wheeler et al., 2025) filters the towns sequentially within each iteration, so an iteration costs about four single-town iterations with the same number of particles in flight. This model tests the panel methods used in Appendix D. Its speedup is lower than for the London model because R runs it relatively fast, likely because the random variate generators in R can take a cheaper route for the smaller parameter values of the smaller towns.

S&P 500. The stochastic volatility model of Bretó (2014) has 3,523 daily log returns and takes one step of its process model per observation, with a single normal draw. Each step between resamples is very cheap, so fixed per-observation costs such as resampling dominate.

Table 2 extends Table 1 with particle filter times and Pypomp on the same 36-core CPU node as R. Pypomp on the CPU is faster than R for every model.

Table 2: Wall-clock time in seconds for IF2 and for particle filter (PF) evaluation of the IF2 estimates. Each IF2 row runs 36 searches from the same starting points on every platform, and Pypomp PF times include JIT compilation. Speedup is the R time divided by the Pypomp GPU time. Cores matched and Peak are as in Table 1. The 4-town panel has no PF comparison because the quant scripts filter different numbers of parameter sets, and it has no Pypomp CPU run.

Model	Task	Pypomp		Pypomp (1 GPU)	Speedup	Cores matched	Peak
		R (36 cores)	(36 cores)				
Cholera, Dhaka	IF2	1,566	390	28.8	54×	1,959	3,152
	PF	359	67.3	8.1	44×	1,590	—
Measles, London	IF2	2,923	1,071	32.2	91×	3,264	5,499
	PF	895	302	14.5	62×	2,223	—
Measles, 4-town panel	IF2	8,187	—	98.2	83×	3,003	3,917
	PF	67.4	—	33.7	—	—	—
S&P 500	IF2	703	275	34.8	20×	727	8,659
	PF	36.4	7.4	5.2	7×	251	—

Appendix B. GPU scaling tests

The benchmark in Table 1 uses the R configuration, which does not give the GPU enough work to reach its highest throughput for every model. The scaling tests in quant ran IF2 on one RTX PRO 6000 Blackwell Server Edition GPU with more particles in flight, added in two ways: more searches of 1,000 particles each, or more particles for each of a fixed number of searches. Each configuration ran in its own process and ran a short IF2 twice, so that the second run excludes JIT compilation. We project each configuration’s time to the benchmark’s number of IF2 iterations, including compilation, and divide the benchmark’s R time by it to get the number of CPU cores running R that one GPU matches. This assumes that R’s time grows linearly with the number of particles and searches. The highest of these values is the Peak column of Table 1. JAX’s memory preallocation was turned off, so the peak GPU memory is the memory each configuration used. No scaling test used more than 1.8 GB of GPU memory, the peak for the cholera model with 2.4 million particles.

Figure 1 shows the results. For every model, throughput peaks at between 1.0 and 2.0 million particles and falls at larger sizes, where the time per iteration grows faster than the number of particles. The peak is 1.6 times the benchmark for the cholera model, 1.7 times for the London measles model, 1.3 times for the 4-town panel, and 11.9 times for the S&P 500 model. The S&P 500 model gains the most because its benchmark, with cheap steps and few particles, leaves the GPU mostly idle. For this model, adding searches beats adding particles per search at a given total number of particles by an average of 6%.

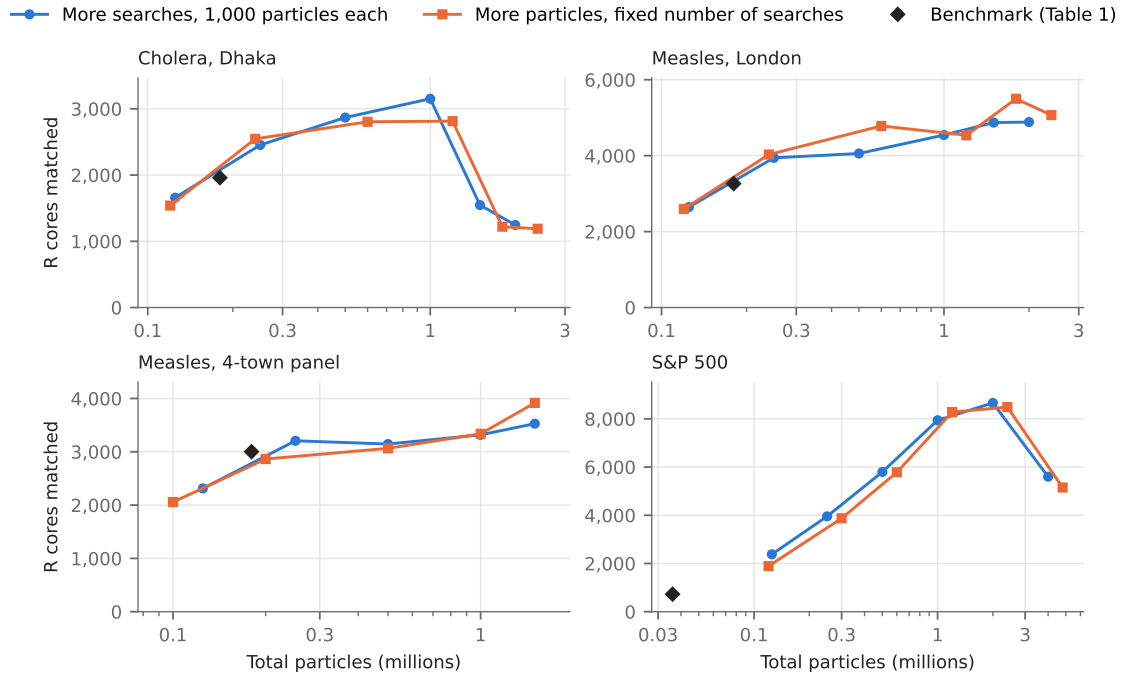


Figure 1: IF2 throughput on one GPU, as the number of CPU cores running R that the GPU matches, against the total number of particles in flight. Blue adds searches of 1,000 particles each; orange adds particles to each of 100 or 120 searches. Each configuration ran 3 or 10 IF2 iterations, projected to the benchmark’s number. The x-axes use a log scale.

Table 3: Time in milliseconds for 1,000,000 draws from `pypomp.random` and `jax.random`, on one GPU and on one CPU core, with parameters spread across the values where the samplers switch algorithms. `jax.random` has no negative binomial sampler. Max W_1/σ is the largest Wasserstein distance between `pypomp.random` draws and draws from `jax.random` (or `scipy.stats` for the negative binomial) over a grid of parameter values, relative to the standard deviation.

Distribution	GPU			CPU			Max W_1/σ
	<code>pypomp</code>	<code>jax</code>	Speedup	<code>pypomp</code>	<code>jax</code>	Speedup	
Poisson	0.03	1.05	35×	37.17	1,321	36×	0.010
Binomial	0.02	1.07	44×	24.56	1,349	55×	0.010
Gamma	0.03	0.76	23×	118	3,046	26×	0.011
Negative binomial	0.07	—	—	187	—	—	0.009

Appendix C. Random variable samplers

`jax.random` draws Poisson, binomial, and gamma random variables with rejection samplers, which loop a random number of times per draw. This suits GPUs poorly, since threads that run together wait for the slowest one. `pypomp.random` instead provides approximate inverse-CDF samplers, which avoid rejection loops. Table 3 compares the two on 1,000,000 draws per call, with parameters spread across the values where each sampler switches algorithms, averaged over 20 calls. The approximation error is small: over a grid of parameter values, the Wasserstein distance between 1,000,000 draws from each `pypomp.random` sampler and from a reference sampler is at most 1.1% of the reference standard deviation. In the London measles benchmark, the `jax.random` samplers made IF2 46 times slower and particle filtering 36 times slower on the GPU.

Appendix D. England and Wales measles fit

We ran 12 MPIF searches, each with 10,000 particles for 200 iterations, and evaluated each endpoint with 36 particle filter replicates of 5,000 particles. We then combined the best estimate for each town across searches and evaluated the result with another 36 replicates. Table 4 gives the time for each stage on one GPU with Pypomp 1.0.0.

For comparison, we estimated how long the same computation would take with `panelPomp` on a 36-core node. We measured `panelPomp`’s cost per particle, per IF2 iteration, and per town on the 4-town panel in Table 2, which covers the same period with the same daily time step, and scaled it linearly to the full computation. We assume `panelPomp` fits the towns jointly with MPIF, as Pypomp did, so each search runs on one core and does not parallelize across units. By this estimate, R would need about 4.5 months, 114 times longer than the GPU, and about 4.8 years on a single core, so the GPU delivered the throughput of about 1,439 CPU cores running R.

Table 4: Wall-clock time for fitting the measles model to all towns in England and Wales. Pypomp times were measured on one GPU. R times are estimates for `panelPomp` on a 36-core CPU node, extrapolated from the 4-town panel in Table 2.

Stage	Pypomp (1 GPU)	R <code>panelPomp</code> , 36 cores (estimated)
IF2	27.3 hours	134.7 days
PF, all search endpoints	99 minutes	3.3 days
PF, combined estimate	10 minutes	6.7 hours
Total	29.1 hours	138.3 days